



电子科技大学
格拉斯哥海南学院
Glasgow College Hainan, UESTC

UESTC 4014: Real-Time Computing Systems and Architecture

Fall Semester 2026-2027

Lab 2 - Week 2

Report Due Date: Wednesday, 30th Sept 2026, 23:59 CST

Student's Name	
Student's UoG ID	
Student's UESTC ID	
Date	

1. Introduction

Most of the time, microcontrollers are employed to interact with external components and other microcontrollers. This can be done by different communication methods and signals like Pulse Width Modulation (PWM). In this lab, you will learn and implement some of these communication protocols, how they work and how they can be employed to communicate with other controllers and external components.

This lab will focus on

- Communication
 - Universal Asynchronous Receiver Transmitter (UART)
 - Inter-Integrated Circuit (I²C)
 - Serial Peripheral Interface (SPI)
- Pulse Width Modulation (PWM).

2. Communication

Communication between different computing devices is essential in different systems either to send different values from one part of the system to another or debug the system, and it is crucial to select the right protocol for the right task. The communication used can be either parallel or serial, as considered in Figure 1.

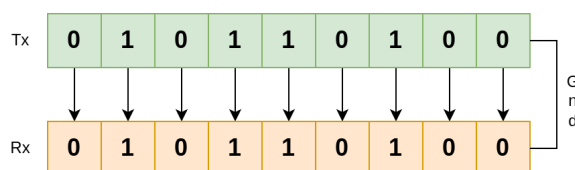


Figure 1 a: Parallel Communication

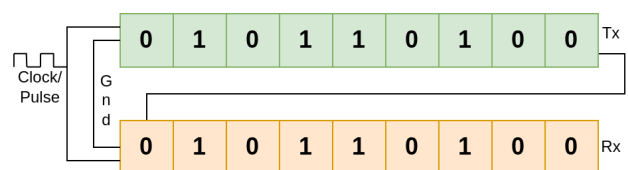


Figure 1 b: Serial Communication

Figures 1(a) and 1(b) show a simple working of parallel and serial communication, respectively. They show the transmission and receiving of 8 bits of data between two devices. In Figure 1(a), although the transmission seems to be really simple and easy as each bit from one side is connected to the respective bit on the receiver bit, it requires 8 lines for the data transmission and 1 line for the ground, meaning 9 lines. Consider the transmission of 32-bit data, requiring 33 lines, which is not feasible and practical. Hence, modern microcontrollers use serial communication protocols. As shown in Figure 1(b), the communication requires only one line for the data transfer, no matter the size. The only downside here is that serial communications require more time as compared to parallel communication as each bit is transferred one by one, and it also requires some clock, signal pulses or both for the transmitter to tell the receiver that it is sending the bits or it has stopped or started sending the data and vice versa. Although there are a plethora of serial communication protocols in microcontrollers ranging from wired to wireless, single device to multiple device and one way to dual way communication, we will have a look at three of the most commonly used protocols.

2.1 Universal Asynchronous Receiver Transmitter (UART)

UART stands for Universal Asynchronous Receiver and Transmitter. It is the simplest communication protocol, which uses two lines i.e., the receiver (Rx) and Transmitter (Tx) lines, to

transmit and receive the data between 2 devices. The UART is an Asynchronous communication protocol, meaning that it does not require any clock, but rather uses pulses to indicate whether the data is to be received or sent. It is mostly used for debugging and communicating with different peripherals like Bluetooth, various sensors, GPS modules, etc.

For the UART communication protocol, you need to declare some of the parameters before the communication starts. These include:

1. Data length: The length of the data that is to be transferred.
2. Start Stop Signal: The signal communicating to the Rx that the data is being sent or has stopped.
3. Baud rate: The speed per second at which the data is to be transferred.

To understand how the UART communication works, imagine we are to transfer decimal number 182 from the Tx to Rx, which is presented in Figure 2.

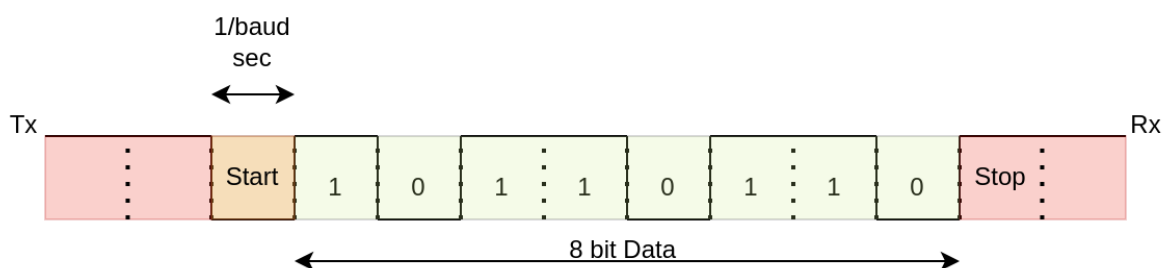


Figure 2: Basic Working of UART

The decimal number 182 can be represented in binary as 0b10110110, which is an 8-bit number, so we define that the data to be transferred is 8-bits in size. Let's say the baud rate is 9600 i.e., 9600 pulses will be transferred every one second, or one pulse will be transferred every $104 \mu\text{sec}$. Now the data length has been defined, and the rate is also defined, but how does the receiver know that it is to start receiving or stop receiving? This is done by the start-stop pulse. Normally the Tx only sends the high signal, but when it is to send some data, it sends a low pulse, for a time period of $\frac{1}{\text{baudrate}} \text{ sec} = \frac{1}{9600} \text{ sec} = 104 \mu\text{sec}$ in our example, and then starts to send the actual data. The receiver considers that new data is being received every $104 \mu\text{sec}$ for a total of $8 \times 104 \mu\text{sec}$ after which the transmitter sends a high signal indicating that all the bits have been transferred and the receiver can stop receiving. So according to Figure 2, the receiver will receive 1 after $104 \mu\text{sec}$ of start bit, then 0 after $104 \mu\text{sec}$, then 1 after $104 \mu\text{sec}$ and so on until all 8 bits are done, and the signal is again high.

2.1.1 UART in STM32

STM microcontrollers use USART communication instead of UART. USART stands for universal synchronous/asynchronous receiver transmitter, meaning that communication can be done in synchronous or asynchronous mode. For this lab, we will be using it in asynchronous mode as discussed in the above examples. Usually, STM32 boards have more than one USART port for example, the NUCLEO-f103rb board being used for this lab, has three different USART ports, and each USART port has its own Rx and Tx pins. For this lab we will consider the USART2 port for STM32 which is already enabled by default, it is the same USART port that is connected to the USB port and used to burn the code in the controller. To have a look at the settings of USART2, open the graphical window and on the left side, under connectivity, select USART2. You will be able to see the mode as Asynchronous and in the parameters settings, the Baud rate to

be 115200 Bits/s, length to be 8 and stop bit to be 1 i.e., when there is no communication the stop bit is high. This is shown in Figure 3.

Now let's try to write a short code in STM32 to display "hello world" and the number of times it is being printed on a serial port. Save the above configurations, which will generate the code activating the serial monitor. If you scroll down, you will be able to see UART2 enabled. The function used by the HAL API to spit the data onto a UART port with the required parameters is given in the following format.

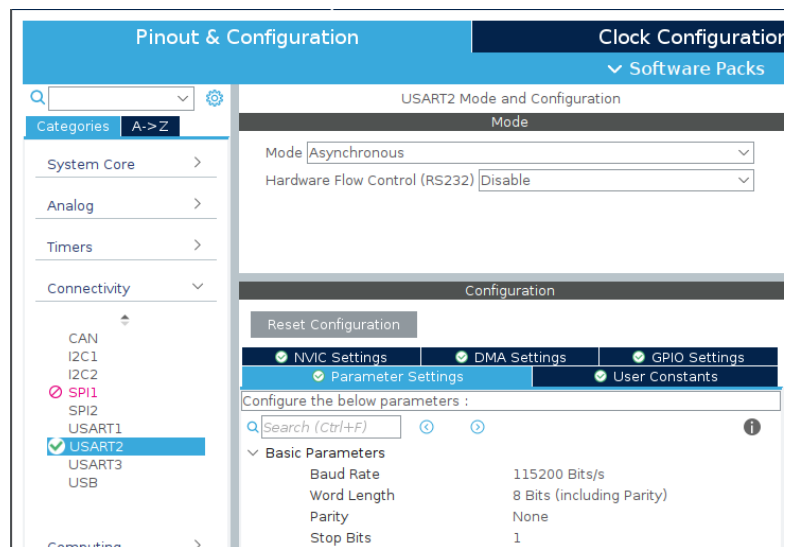


Figure 3: USART Configuration

`HAL_UART_Transmit(&uart_port, (string_pointer*)string, string_lenght, DELAY);`

The function `HAL_UART_Transmit()` is used to transmit data from one of its serial ports. The `&uart_port` is the UART port in use. `String_pointer` is the pointer to the character array or the string that is required to be transmitted, `srting_length` is the length of the string, and `DELAY` is some amount of time to ensure that the data has been transferred.

The following code snippets can be used to write the above-mentioned code. Do not forget to include the `string.h` and `stdio.h` as we are using strings.

```
#include "main.h"

/* Private includes -----
/* USER CODE BEGIN Includes */
#include <string.h>
#include <stdio.h>
/* USER CODE END Includes */
```

Figure 4 a: Include libraries

```
/* USER CODE BEGIN 1 */
int num=0;
char char_str[] = "hello world";
char num_str[10];
/* USER CODE END 1 */
```

Figure 4 b: variables

```
while (1)
{
    num = num+1; // adding every time a new hello world is printed
    sprintf(num_str, " %hu\n", num); // converting int to string
    // transmitting to the UART PORT 2
    HAL_UART_Transmit(&huart2, (uint8_t*)char_str, strlen(char_str), HAL_MAX_DELAY);
    HAL_UART_Transmit(&huart2, (uint8_t*)num_str, strlen(num_str), HAL_MAX_DELAY);
    HAL_Delay(1000); // delay of a second
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
}
```

Figure 4 c: main loop for UART

Now as mentioned earlier, we can also receive data from another controller using UART, but there are three methods for that in STM32. These are

- Polling method: This is a blocking method, which blocks the CPU until the data is not received
- Interrupt method: This is a more efficient method of receiving data in UART as it does not block the CPU to receive data, instead the CPU keeps on working, but an interrupt is called as soon as some data is received by the Rx and the interrupt service routine (ISR) for that specific interrupt is called.
- Direct Memory Access (DMA) method: DMA stands for direct memory access. In this method, the CPU does not have to do anything, and the data received by the Rx is directly transferred to the memory and the CPU is notified.

In this lab, we will use the interrupt method to receive the data from an external Tx. To do so we can get into the graphical interface. Again, in the USART2 settings, change the baud rate to 9600 Bits/s and in the NVIC settings enable the USART2 global interrupt and save the file. This will generate the code with the USART2 interrupt enabled. The settings are shown in figure 5. Now save the file and the IDE will generate the code with necessary libraries and functions.

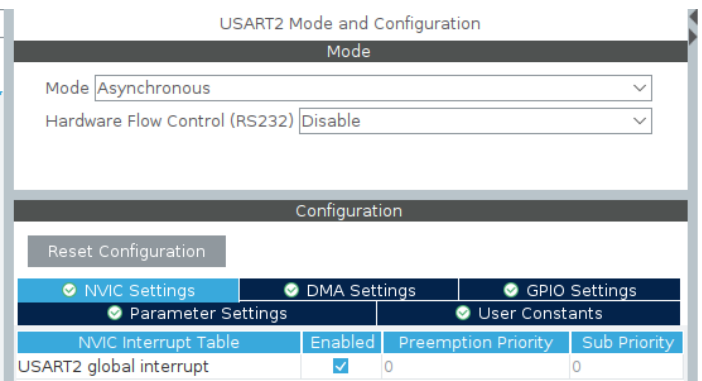


Figure 5: USART 2 Interrupt Enabled

To be able to use the USART in interrupt to receiver data, we need to know the callback function which is called when the data is received and the receiving start function which activated the interrupt. These are as follows

Callback function: void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)

Activate interrupt: HAL_UART_Receive_IT (&huart, rx buffer, no. of bits)

Here *&huart* defines the USART being used, *rx_buffer* is the variable which will store all the data that is received, and the no. of bits is the number of characters that will be received. To better understand how to receive data lets have a look at an example where you will send your student ID serially and your ID will be printed on the serial monitor. Open the main.c file and make a variable of the type *uint8_t* named *rx_data[10]*, which will store the data received this is shown in figure 6a. Now start the interrupt using the interrupt activation function. This is shown in figure 6b. One important thing that is to be noted here is that make the number of characters you want to send to be 1 less than the number of bits. Hence of example if your ID is 2726660z, the number of bits should be set to 9. Every time the data is received and the control goes into the ISR, the interrupt needs to be reset. This can be done in the ISR itself. The ISR is shown in figure 6c.

```
/* Private variables -----
UART_HandleTypeDef huart2;
uint8_t rx_data[10];
/* USER CODE BEGIN PV */
```

Figure 6a: RX Buffer

```
/* USER CODE BEGIN 2 */
HAL_UART_Receive_IT (&huart2, rx_data, 9);
```

Figure 6b: Interrupt activation

```

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    HAL_UART_Transmit(&huart2, (uint8_t*)rx_data, strlen(rx_data), HAL_MAX_DELAY);
    HAL_GPIO_TogglePin (GPIOA, GPIO_PIN_5);
    rx_data[0]='\0';
    HAL_UART_Receive_IT (&huart2, rx_data, 9); // restart the interrupt reception mode
}

```

Figure 6c: Call Back Function

Now overall what happens is that initially, in the main function, the interrupt is activated and the control goes in the while(1) and whatever code is in the while loop keeps on executing, as soon as the bits are received, they are put in rx_data and the callback is called. In the callback function, the data in the rx_data is again sent to the serial monitor, the LED is toggled, the rx_data is emptied and the data and the USART interrupt is again activated.

TASK 1: Write a project to take three 2-digit numbers from the user, ask them whether they want to add them or multiply them and print out the answer on the serial monitor. Copy the code with comments, in your report and show the screen shots of your output, for addition and multiplication. Also briefly explain how your code works. [25 marks]

2.2 Inter-Integrated Circuit (I²C)

I²C was the first developed synchronous communication developed by Philips and require 2 communication wires, called SDA and SCL. The SDA is the data pin whereas the SCL is the clock pin which provides the system with a clock and hence makes the system synchronous. Once again, the number of bits that are to be sent or received are to be specified. Usually, I²C can work with speeds up to 400 kilobits per second. Consider the figure 7 which shows an I²C setup of a master with different slaves.

As can be seen in the figure 7, the master sends the SCL (clock pulses) with the same frequency as SDA (data), which enables the slaves to know when a bit ends and hence does not have to count the bits making the communication process to be carried out faster. But now the question arises that how the specific slave knows if the

master is communicating with it or some other slave. This is done by specifying the slave address. Every slave has a specific address and before starting the actual communication, the master specifies the slave address with which it wants to communicate and then sends the actual data.

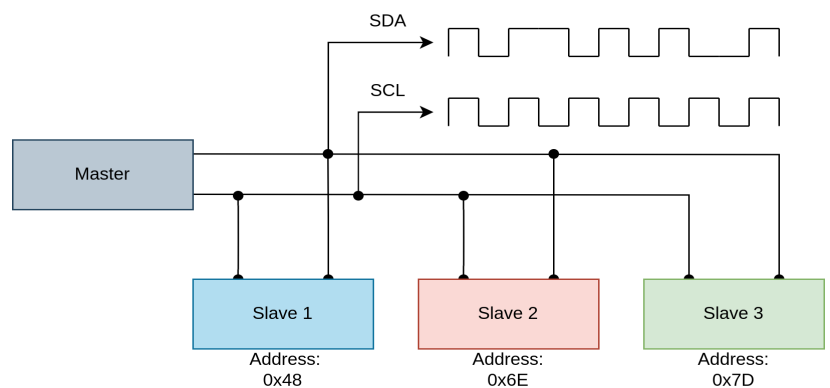


Figure 7: I²C block diagram

To understand the working and the components of the I2C communication consider figure 8a and b.

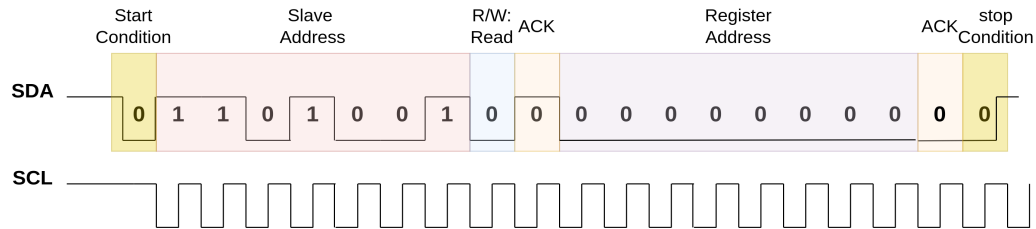


Figure 8a: I2C Working 1

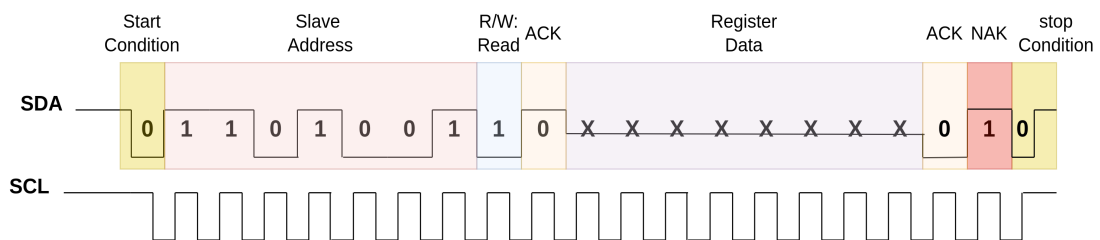


Figure 8b: I2C working 2

Let's consider we are dealing with an I²C slave which can take the address of 8-bit registers and return an 8-bit data in that register.

I²C Working 1:

Start Condition: This is where the master pulls the SDA line low, while the SCL line is high and indicates that it is about to start the communication.

Slave address: Slave address is the address of the slave with which the master wants to communicate. After this address is sent, the rest of the data bits will only be received by that specific device and the rest of the devices will ignore the data. In the figure 8a, the address is 0b1101001 or 0x6E hence in case of figure 7, slave 2 will communicate with the master.

R/W bit: This bit indicates whether the master wants to read (R/W bit = 1) from or wants to write to (R/W bit = 0) the slave device. In the case of figure 8a the master wants to write to the slave.

ACK: ACK bit is the acknowledgment bit which is set by pulling the SDA pin low by the target device acknowledging that it has received the byte.

Data/Register: After the ACK bit is received by the master it sends the data to the slave. In the case of the figure 8a, the master sends the address of the register it wishes to read. After the register byte is received, the slave again pulls the SDA line low, indicating that that the address byte has been received.

Stop Condition: The master after receiving the ACK bit again stops the communication but setting the SDA pin high just after the SCL line becomes high.

I²C Working 2:

In the figure 8b, the master again initializes the communication with start condition and sends the address of the slave it wants to communicate with, which again in this case is 0x6E but this time

it sends the R/W bit to 1 indicating that it wants to read from the register it sent earlier. The slave sends the ACK acknowledging that it has received the byte and starts sending back to the master. In the figure 8b, this data is represented by x as it can be anything. Every time a byte is received, the master acknowledges that the byte has been received. Once all the required data is received, the master pulls the SDA line high giving a Negative Acknowledging bit NACK indicating that all the data has been received and stops the communication by pulling stop condition.

2.2.1 I²C in STM32

In STM32 IDE you do need to worry about all this procedure rather the HAL library managed everything for you. To communicate with another I²C device you can use the following commands to transmit and receive the data respectively.

```
HAL_I2C_Master_Transmit(&hi2c#, I2C_address, buff, #_of_bytes,DELAY);
HAL_I2C_Master_Receive(&hi2c#, I2C_address, buff, #_of_bytes,DELAY)
```

In the above commands, the &hi2c# is the i²c used. The nucleo board provides with i²c 1 and 2 which can be adjusted according to the developer's needs. The I2C_address is the address of the slave with which the communication needs to be developed. Buff is the buffer array through the data is sent and in which the data received is stored, and #_of_bytes is the number of bytes STM32 expect to receive. DELAY is the amount of time to consider a communication session to terminate.

To better understand how it works let's consider an example with the Arduino nano provided. The Arduino is programmed such that according to certain bytes sent to it, it will do specific tasks. These are as follows:

Value	Action
0x83	Blink built in LED once a second
0x82	Blink built in LED once 75 ms
0x81	Blink built in LED once a 500 ms
0x80	LED to remain on
0x00	Address of analog value measured by Arduino
0x01	Address of 4 bytes of message in the Arduino

Table 1: Messages and actions

To make everything to work, make the connections as shown in the Figure 9. The Pin connection is as follows

```
Nucleo SCL => ARDUINO A5
Nucleo SDA => ARDUINO A4
Nucleo GND => GND
Variable Resistor out => Arduino A1
```

Now, set up the IDE to enable us to use the I²C communication protocol. Create a new project and in the graphical interface, click the PB9 and PB8 and from the drop-down menu, select them as I2C1_SDA and I2C1_SCL respectively. Then from the connectivity section, as we are going to use

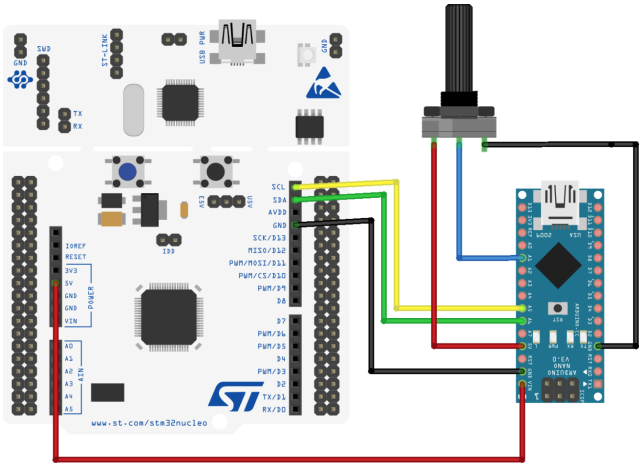


Figure 9: I²C communication with Arduino

I2C1, select I2C1, and select I2C in the mode. This is shown in Figure 10.

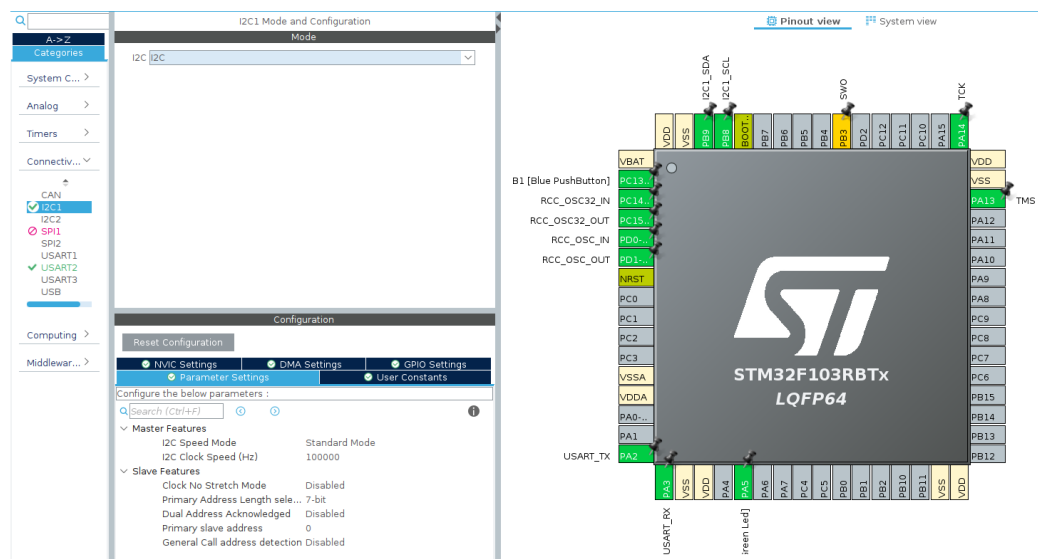


Figure 10: I²C communication setup

Now that we have the communication setup done, let's dive into the code. Make sure that all the needed files are included under the main.h, these are string.h and stdio.h. First, define some variables and constants needed. These are shown in Figure 11a.

```
int main(void)
{
    /* USER CODE BEGIN 1 */

    HAL_StatusTypeDef ret;
    static const uint8_t nano_addr = 0x55 << 1; // left shifting as bit0 used to indicate read write
    uint8_t buff[12]; // Buffer to receive and send 8 bit data
    uint8_t cmd_buff[12]; // buffer save the data when a command sent
    uint8_t analog_buff[12]; // buffer to save analog data
    uint8_t message_buff[12]; // buffer to save the message received
    uint16_t analog_val; // 16 bit analog value
    uint8_t cmd_0 = 0x83; // send to make the LED blink once a second
    uint8_t cmd_1 = 0x82; // send to make the LED blink once 750 ms
    uint8_t cmd_2 = 0x81; // send to make the LED blink once 500 ms
    uint8_t cmd_3 = 0x80; // send to make the LED to remain on

    uint8_t REG_0 = 0x0; // receive analog value
    uint8_t REG_1 = 0x1; // Receive the message
}
```

Figure 11a: Variables and Constants

In the above, one thing is to be noted that, the constant *nano_addr* is the address to the Arduino nano slave, but while saving it to the constant, it is shifted left by one. The reason behind that is that, as mentioned in the explanation of the I²C communication above, the least significant bit is used to indicate whether to read from the slave or to write to the slave, hence this format. Another variable to be noted is the *HAL_StatusTypeDef* type *ret*, which is a flag indicating whether the communication is a success or not.

Now, scroll down and try to make the Arduino LED blink once every 750 ms. In the *while(1)* loop we see the code snippet, shown in Figure 11b.

Here initially, the `cmd_1` is added to the 0 position of the `buff` array, and then in the next line, the communication is started. `ret` is used to store the status of the communication. As here some data is to be transmitted or written to the

```
// sending the command to adjust the light blinking speed
buff[0] = cmd_1; // adding blinking command to the buffer
ret = HAL_I2C_Master_Transmit(&hi2c1, nano_addr, buff, 1, HAL_MAX_DELAY);
if (ret != HAL_OK) // if communication not successful
{
    // printing error on the serial monitor
    strcpy((char*)buff, "Error Rx\r\n");
    HAL_UART_Transmit(&huart2, buff, strlen((char*)buff), HAL_MAX_DELAY);
}
// resetting buffer
memset(buff, '\0', sizeof(buff));

// delay of 2 seconds
HAL_Delay(2000);
```

Figure 11b: Making the LED blink

slave, the `HAL_I2C_Master_Transmit` is used. Here in the arguments, initially it is stated which I2C is to be used, as mentioned I2C1 is to be used so `&hi2c1`, then the address of the slave is stated, which is stored above in the `nano_addr` 8-bit constant. Then the address of `buff`, where the command is saved, the number of bytes to be passed i.e. 1 and lastly the delay for the STM to wait which here is kept to be `HAL_MAX_DELAY`. Now if the communication is successful, `ret` status will be okay and the `buff` is cleared, but in case, the communication is not successful, a message is received on the serial monitor, indicating a problem.

This is how the transmission can be done. Now, have a look at how we can receive data from the slave and give it its actual shape. You can try to get the analog value measured by the Arduino nano. The maximum value that the Arduino can measure in this case is 1024 which is stored in two 8-bit registers or bytes. Hence to receive the complete data, 2 bytes must be received. Have a look at the code snippet for this shown in Figure 11c.

To receive the analog data from the arduino, before sending receive command, you need to transmit what data is required by you. Hence initially the `buff` is stored with the `0X00` value, indicating the value of register 0 is required, which stores the analog value measured by the arduino. Once this command is transmitted and

```
// writing the address of the data required i buffer
buff[0] = REG_0;
ret = HAL_I2C_Master_Transmit(&hi2c1, nano_addr, buff, 1, HAL_MAX_DELAY);

if (ret != HAL_OK) // if communication not successful
{
    // printing error on the serial monitor
    strcpy((char*)buff, "Error Rx\r\n");
    HAL_UART_Transmit(&huart2, buff, strlen((char*)buff), HAL_MAX_DELAY);
}
else // if communication successful
{
    HAL_Delay(500); // delay
    // receiving 2 bytes via i2c
    ret = HAL_I2C_Master_Receive(&hi2c1, nano_addr, buff, 2, HAL_MAX_DELAY);
    if (ret != HAL_OK) // if communication not successful
    {
        // printing error on the serial monitor
        strcpy((char*)buff, "Error Rx\r\n");
        HAL_UART_Transmit(&huart2, buff, strlen((char*)buff), HAL_MAX_DELAY);
    }
    else // if communication successful
    {
        // converting the data received into the actual value
        analog_val = buff[1];
        analog_val = buff[0] | (analog_val << 8);
        sprintf(analog_buff, "%hu\n", analog_val);
        HAL_UART_Transmit(&huart2, analog_buff, strlen((char*)analog_buff), HAL_MAX_DELAY);
    }
}
memset(buff, '\0', sizeof(buff)); // resetting buffer
HAL_Delay(2000); // delay
```

Figure 11c: Sending and Receiving data 1

received by the Arduino. The Arduino gets ready to transmit the analog value when a receive request is sent by the STM32. So once the initial transmission is successful, the STM sends the

receiving command using the `HAL_I2C_Master_Receive` function. In the code above, the argument `buff` indicates that the data received will be stored in the `buff` buffer and the number of bytes are 2. Hence the first byte received will be stored in `buff[0]` and the second byte will be received in `buff[1]`.

Now the question arises how to convert this data back to the original analog data. The variable `analog_val` is a 16-bit variable and by the help of a little bitwise manipulation the data is converted back into its original form and again printed on the terminal. And this is how the data can be received.

TASK 2: Upload the arduino code provided to you in your arduino nano. It has the same commands and address values as stated in the above examples. Write a project to take 2 inputs from the user via UART and according to these inputs, adjust the speed with which the arduino LED blinks and the data which is to be received. View this received data on the serial monitor. Copy the commented code in your report with screen shots of the data received once for when the command sent to the arduino is 0x00 (for the 2 byte analog value) and once for 0x01 (4 byte data). Also briefly explain how your code work. [25 marks]

2.3 Serial Peripheral Interface (SPI)

SPI is a synchronous communication protocol developed by Motorola and is a defacto standard meaning there is no formal standard for it and hence one can run in multiple variations for naming and implementation by different companies and hence whats explained here might be different what you might come across in some other application but more or less if you understand the basics of how SPI works, you will be able to make things work in other applications as well by some effort and the data sheets provided.

To understand how SPI works, have a look at Figure 12 a.

The figure shown below offers a full duplex communication meaning that both the master and the slaves can send and receive data. Here CLK is the clock signal, MOSI is master out slave in i.e. the pin from where master transmits the data, and the slave receives the data. MISO stands for master in slave out. The CS1 and CS2 are the chip select pins. These pins are used to determine which slave the master wants to establish the communication with. It should be noted that the chip select pins are active low.

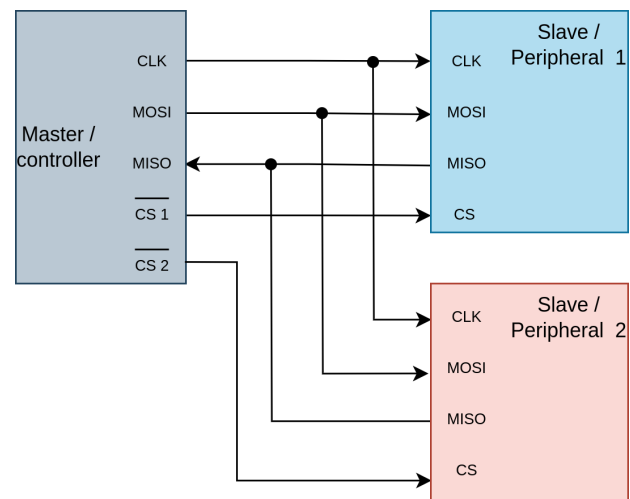


Figure 12a: SPI block diagram

To better understand SPI and how its pins work, check the clock diagrams of these pins shown in Figure 12b.

According to the figure 12b, the communication between slave1 starts when the CS1 goes low. Here the data is recorded at the rising CLK signal. One thing to be noted is that the communication shown is 8-bit, i.e. 8 bits or 1 byte is communicated at a time. This can be adjusted depending upon the slave Peripheral in use. For example, if communicating with a peripheral that sends 16 bits data instead of 8 bits at a time, 16-bit communication can be used. Another important point is that both the master and the slave are transmitting and receiving data between each other simultaneously unlike i2c, where only one way communication can be done at a time.

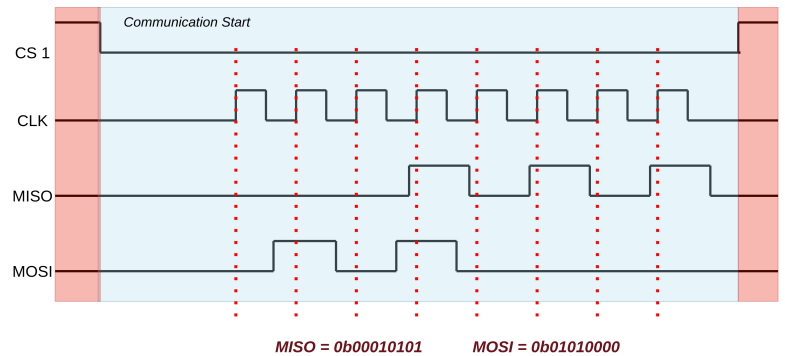


Figure 12b: SPI block diagram

2.2.1 SPI in STM32

For SPI, the HAL API provides different SPI functions to perform the SPI communication between STM32 controllers and Peripherals. The functions used to transfer data through MOSI and MISO are as follows, respectively.

```
HAL_SPI_Transmit(&hspi#, buffer, # of bits, delay);
HAL_SPI_Receive(&hspi#, buffer, # of bits, delay);
```

In the above commands, the *&hspi#* is the SPI used. The nucleo board provides with SPI 1 and 2 which can be adjusted according to the developer's needs. The Buff is the buffer array through the data is sent and in which the data received is stored, and *#_of_bytes* is the number of bytes STM32 expect to receive. DELAY is the amount of time to consider a communication session to terminate. The functions are quite similar to I2C commands, but one thing to be noted is that there is no address for the Peripheral to be used, because this is done by the IO pins, and selecting them low to activate the communication and high to stop the communication.

We can consider an example to better understand how to communicate between STM32 and another peripheral via SPI. The Arduino NANO will be used as a peripheral here. The table 2 shows what the Arduino will do if any specific byte is sent to it.

Value	Action
0x83	Blink external LED once a second
0x82	Blink external LED once 750 ms
0x81	Blink external LED once a 500 ms
0x80	Blink external LED once a 250 ms
0x00	address of lower byte of analog value ready to sent
0x01	address of high byte of analog value ready to be sent

Table 2: SPI Messages and actions

For the SPI communication to work, make the connections as shown in Figure 13. The Pin connection is as follows

Nucleo SCK => ARDUINO D13
 Nucleo MOSI => ARDUINO D12
 Nucleo MISO => ARDUINO D11
 Nucleo CS => Arduino D10
 LED Anode => Arduino D2
 Variable Resistor out => Arduino A1

Open the graphical interface and go to SPI menu in the communication. In there select SPI 1 and enable the mode to be Full-Duplex as we are going to send and receive from the Arduino. Select the prescaler to be 128. Select the PB6 to be output as this it is going to be our CS. These settings are shown in Figure 14.

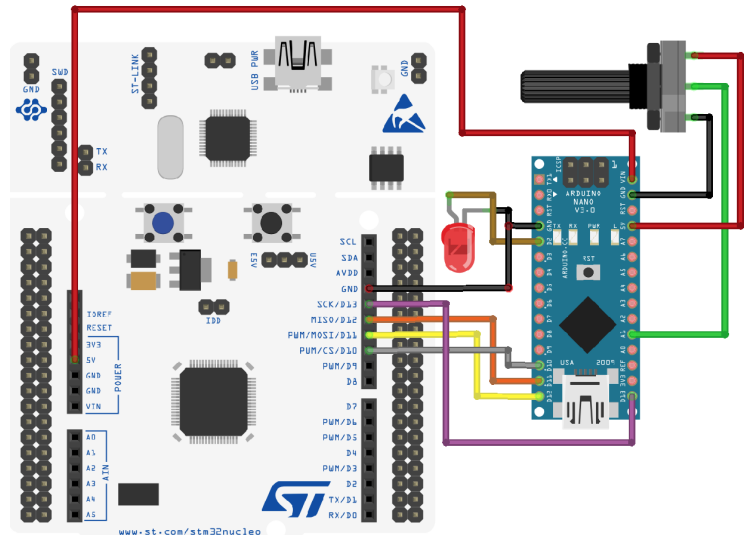


Figure 13: SPI Communication with Arduino

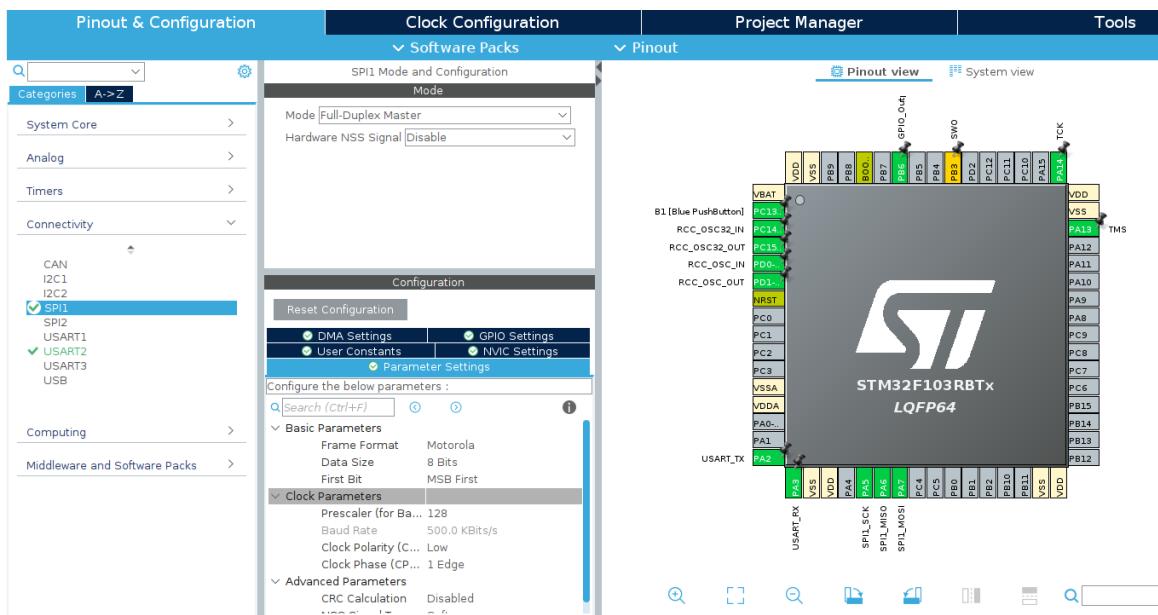


Figure 14: SPI Communication Setup

Save the file and the code with essential SPI setup will be generated.

In the code, make sure that all the needed files are included under the *main.h*, these are *string.h* and *stdio.h*. First, we have to define some variables and constants need.

Now let's try to make the LED connected to the Arduino nano blink once every second. As can be seen, we need to send cmd_0 or 0x83 to the Arduino MOSI. To do so, we first make the CS pin

low, which enables the SPI communication and then start to transmit the data, once that is done, we pull the pin high again. As mentioned above, the PB6 is used as the CS pin, it will be pulled low and high. The code is shown in figure 15b. One thing to be noted is that, in the code, the PB6 is initially pulled high before the while loop starts so that the communication doesn't start right away. Although it is not essential to do so, but it is a good practice.

In the figure, in the *while(1)* loop, initially the PB6 is set low to start the communication, after which the buffer is cleaned. Then the cmd_0 i.e. 0x83 is loaded in into the buffer. Now this buffer value is transmitted through the *HAL_SPI_Transmit()* function. After the transmission, the CS pin is set high again which stops the communication. In return, the LED starts to blink once a second.

Now try to retreat the analog value measured by the Arduino. This value is divided into 2 parts an 8-bit lower value and an 8-bit upper value stored in 2 different registers of address 0 and 1 respectively. Sending the register values, sets the register value ready to be received and can be received by the STM32. We can check have a look of how it's done in the Figure 15c.

```
int main(void)
{
    /* USER CODE BEGIN 1 */
    uint8_t buff[12]; // Buffer to receive and send 8 bit data
    uint16_t analog_val; // 16 bit analog value
    uint8_t cmd_0 = 0x83; // send to make the LED blink once a second
    uint8_t cmd_1 = 0x82; // send to make the LED blink once 750 ms
    uint8_t cmd_2 = 0x81; // send to make the LED blink once 500 ms
    uint8_t cmd_3 = 0x80; // send to make the LED to blink once every 250 ms
    uint8_t reg_0 = 0x0; // make the lower byte ready to be transferred
    uint8_t reg_1 = 0x1; // make the higher byte ready to be transferred
    /* USER CODE END 1 */
}
```

Figure 15a: Variables

```
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_SET);
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */

    // making the LED blink once a second
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_RESET);
    memset(buff, '\0', sizeof(buff));
    buff[0] = cmd_0;
    HAL_SPI_Transmit(&hspi1, buff, 1, HAL_MAX_DELAY);
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_SET);
    HAL_Delay(100);
}
```

Figure 15b: Blinking LED (Sending Value)

```
// sending the Address
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_RESET);
memset(buff, '\0', sizeof(buff));
buff[0] = 0x0;
HAL_SPI_Transmit(&hspi1, buff, 1, HAL_MAX_DELAY);
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_SET);
HAL_Delay(100);

// Receiving the Data of the Address sent
memset(buff, '\0', sizeof(buff));
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_RESET);
HAL_SPI_Receive(&hspi1, buff, 1, HAL_MAX_DELAY);
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_SET);
analog_val = buff[0];
HAL_Delay(100);

// sending the Address
memset(buff, '\0', sizeof(buff));
buff[0] = 0x1;
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_RESET);
HAL_SPI_Transmit(&hspi1, buff, 1, HAL_MAX_DELAY);
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_SET);
HAL_Delay(100);

// Receiving the Data of the Address sent
memset(buff, '\0', sizeof(buff));
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_RESET);
HAL_SPI_Receive(&hspi1, buff, 1, HAL_MAX_DELAY);
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_SET);
HAL_Delay(100);

// Converting the value into int
analog_val = (buff[0] << 8) | (analog_val);
memset(buff, '\0', sizeof(buff));
sprintf(buff, "%hu\n", analog_val);

// printing data
HAL_UART_Transmit(&huart2, buff, strlen((char*)buff), HAL_MAX_DELAY);
```

Figure 15c: Getting Analog Data (Sending and

3. PWM

PWM, or Pulse Width Modulation, is a widely used technique in electronics and control systems. It involves rapidly switching a signal between two states: high and low, to control the average voltage or power delivered to a device. By varying the width of the high and low states over time, PWM can effectively regulate the output and achieve precise control over devices such as motors,

LEDs, and heaters. This modulation method is valued for its efficiency, simplicity, and ability to simulate analog signals with digital components, making it indispensable in applications ranging from motor speed control to dimming lights and regulating power in numerous electronic devices. In STM32 devices this is done by the help of timers. Let's Understand by the help of a simple example.

Suppose you want to control the brightness of an LED. This can be done by the help of PWM. The next question that arises is what the frequency of the PWM should be, i.e. the switching rate. Let's say we want the frequency to be 10 Hz.

To create the project according to the given specifications, In the IDE, open the graphical interface and go into the timer section. In TIM4, select the MODE, click the internal clock and select channel 1 and *PWM generation CH1*. The frequency of the PWM is adjusted by the help of the timer frequency and the counter period.

Now in the parameter selection, we are to configure the parameters according to the specifications mentioned above. First configure the timer frequency by selecting the frequency to be 1000 Hz for the ease of calculation. As the internal clock frequency is 64 MHz, select the prescaler to be 64000 - 1 which will make the timer frequency 1000 Hz.

Now select the counter period. The complete counter period is the cycle of the PWM. Now as we want the PWM to be of a frequency of 10 Hz, we select the counter period to be 100. In the PWM Generation Channel 1 parameters, the pulse represents the duty cycle. For example, in the above scenario, if we want the duty cycle to be 30 %, we will select the pulse to be 30. This can be changed in the code as well. The CH polarity represents the period for which the pin is to be high. If high, the pin will be high for the pulse time and low for the rest of the counter. And if low, the pin will be low for the pulse and high for the rest. This is illustrated in figure 16 and the parameter setup is shown in figure 17.

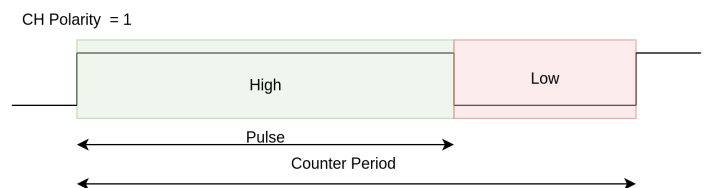


Figure 16: PWM Explanation

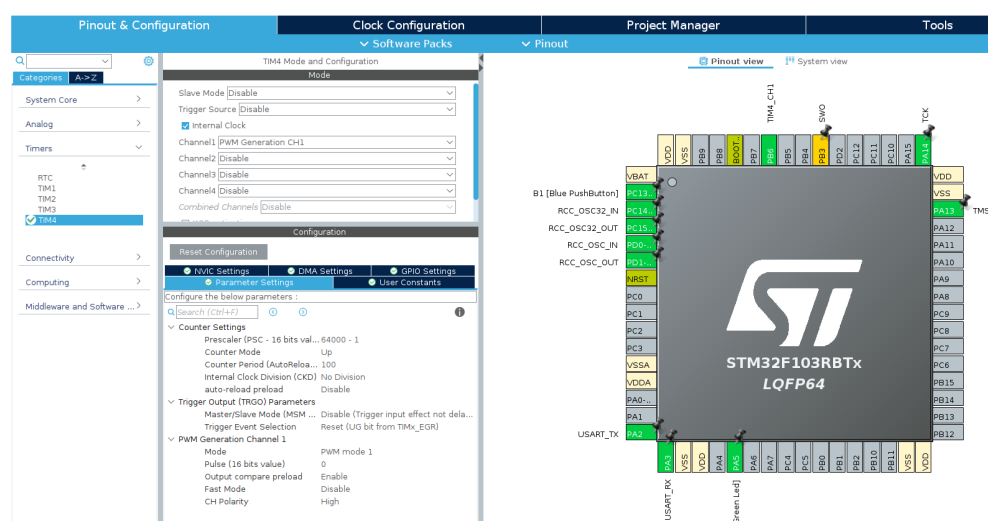


Figure 17: PWM Parameter Setup

One thing to be noted in the figure 17 is that the IO pin for channel 1 of timer 4 is PB6. In this way, each channel of each timer has its own specific pin.

Now dive into the code. According to the code, initially the PWM is started by `HAL_TIM_PWM_Start()`. Then a variable `duty_cycle` is set which will be used to set the duty cycle of our PWM. In the while loop, the `duty_cycle` is increased by 10 after every second and is assigned to the `htim4.Instance->CCR1` which is the pulse for the PWM. Hence the LED will be at its brightest in 10 seconds and will start from 10 again.

```
/* USER CODE BEGIN 2 */
// initilizing the timer4 CHANNEL 1 pwm
HAL_TIM_PWM_Start(&htim4,TIM_CHANNEL_1);
uint16_t duty_cycle = 10;
/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    duty_cycle += 10;
    if (duty_cycle == 100)
    {duty_cycle = 10;}
    // setting Duty cycle in the timer PWM channel
    htim4.Instance->CCR1 = duty_cycle;
    HAL_Delay(1000);
}
/* USER CODE END WHILE */

/* USER CODE BEGIN 3 */
```

Figure 18: PWM Generation Code

TASK 3: Upload the Arduino code for SPI provided to you in your Arduino nano. Receive the analog value through SPI and according to the data received, adjust the PWM of your channel. Provide the commented code in your report. Also show the calculations and discuss how your code is working. [25 marks]

TASK 4: Upload the Arduino code for SPI provided to you in your Arduino nano. Receive the analog value through SPI and according to the data received, adjust the PWM of your channel to rotate the servo motor. The servo motor is rotated according to the PWM provided to it. Provide the commented code in your report. Also show the calculations and discuss how your code is working. [25 marks]